

LAPORAN PRAKTIKUM 3

Review 4 Pillar Object Oriented Programming (OOP)

Nama : Abadan Syakira

NIM : 2024573010099

Kelas : TI 2A

Mata Kuliah : Praktikum Design Pattern

Tanggal Praktikum : 06 April 2026

1. Pendahuluan

1.1 Tujuan Praktikum

- **Memahami Konsep Dasar OOP:** Mahasiswa mampu memahami landasan Pemrograman Berorientasi Objek dalam bahasa Java.
- **Implementasi Struktur Dasar:** Mahasiswa mampu membuat *Class*, mendefinisikan atribut, dan menyusun *Method*.
- **Penerapan Encapsulation:** Mahasiswa dapat mengamankan data menggunakan *Access Modifier* serta mengelolanya melalui *Setter* dan *Getter*.
- **Inisialisasi Objek:** Mahasiswa memahami fungsi *Constructor* dan cara kerjanya dalam menciptakan objek.
- **Studi Kasus Nyata:** Mahasiswa mampu menggabungkan seluruh konsep untuk membangun sistem sederhana (seperti Sistem Manajemen Perpustakaan) menggunakan `ArrayList` dan `Scanner`.

2. Landasan Teori

2.1 Class dan Object

Class adalah sebuah cetak biru (*blueprint*) atau definisi formal yang menentukan atribut dan perilaku yang akan dimiliki oleh objek. Sedangkan **Object** adalah instansi nyata yang dibuat berdasarkan definisi *Class* tersebut.

- Contoh: *Class* adalah "Mobil", sedangkan *Object* adalah "Mobil Toyota milik Budi".

2.2 Atribut dan Method

- **Atribut:** Merupakan variabel di dalam kelas yang menyimpan data atau keadaan dari objek (misalnya: warna, nama, harga).
- **Method:** Merupakan blok kode atau fungsi yang mendefinisikan apa yang bisa dilakukan oleh objek tersebut (misalnya: berjalan, menghitung luas, menampilkan informasi).

2.3 Akses Modifier

Access Modifier digunakan untuk mengatur hak akses terhadap suatu *member* (atribut/method) dalam sebuah program Java agar data lebih terstruktur dan aman.

- **Public:** Member dapat diakses dari mana saja (baik dari dalam kelas, luar kelas, maupun luar package).
- **Private:** Member hanya dapat diakses di dalam kelas tempat ia didefinisikan. Ini digunakan untuk menyembunyikan detail internal (Data Hiding).

2.4 Setter dan Getter

Karena atribut sering kali bersifat `private`, maka diperlukan method khusus untuk berinteraksi dengan data tersebut:

- **Getter:** Method publik yang berfungsi untuk mengambil atau membaca nilai dari atribut privat.
- **Setter:** Method publik yang berfungsi untuk mengisi atau memperbarui nilai atribut privat, yang memungkinkan adanya validasi data sebelum disimpan.

2.5 Constructor

Constructor adalah method khusus yang dieksekusi saat sebuah objek baru diciptakan. Namanya harus sama dengan nama kelas dan tidak memiliki tipe data pengembalian (*return type*).

- **Default Constructor:** Constructor tanpa parameter yang biasanya memberikan nilai awal standar pada atribut.
- **Parameterized Constructor:** Constructor yang memiliki parameter untuk langsung menginisialisasi atribut dengan nilai spesifik saat objek dibuat.
- **Constructor Overloading:** Praktik menyediakan lebih dari satu constructor dengan daftar parameter yang berbeda dalam satu kelas yang sama untuk memberikan fleksibilitas pembuatan objek.

3. Analisis Praktikum

Berikut merupakan analisis terhadap program-program utama yang telah dikerjakan pada praktikum:

3.1 bagian_1

Mahasiswa.java

```
package modul_2.bagian_1;

public class Mahasiswa {
    String nama;
    int umur;
}
```

Main.java

```
package modul_2.bagian_1;

public class Main {
    public static void main(String[] args) {
        // Membuat object dari class Mahasiswa
        Mahasiswa mhs1 = new Mahasiswa();

        // Mengisi nilai atribut
        mhs1.nama = "Budi";
        mhs1.umur = 20;

        // Menampilkan nilai atribut
        System.out.println("Nama: " + mhs1.nama);
        System.out.println("Umur: " + mhs1.umur);
    }
}
```

Analisa

Kode ini menunjukkan konsep paling fundamental dalam Pemrograman Berorientasi Objek, yaitu pembuatan sebuah blueprint dan perwujudannya dalam bentuk objek. Pada class Mahasiswa, atribut nama dan umur dideklarasikan menggunakan akses default (package-private), yang berarti variabel-variabel tersebut dapat diakses secara langsung oleh kelas lain selama masih berada dalam satu folder atau package yang sama. Struktur ini sangat sederhana karena hanya berfungsi sebagai wadah penyimpanan data mentah tanpa adanya logika pemrosesan di dalamnya. Di dalam class Main, proses instansiasi dilakukan dengan keyword new untuk menciptakan objek mhs1 dari cetak biru Mahasiswa. Pengisian data dan pemanggilan nilai dilakukan melalui akses langsung ke atribut menggunakan operator titik (.). Analisis terhadap alur ini menunjukkan bahwa meskipun program berhasil berjalan dan menampilkan data ke layar, terdapat celah dalam sisi keamanan data atau enkapsulasi. Karena atribut dapat diubah secara bebas dari luar class, tidak ada mekanisme kontrol untuk memvalidasi apakah data yang dimasukkan (seperti nilai umur) sudah benar atau masuk akal. Hal ini menjadi dasar pentingnya pengembangan kode ke tahap enkapsulasi menggunakan akses modifier di bagian praktikum selanjutnya.

3.2 bagian_2

Kalkulator.java

```
package modul_2.bagian_2;

public class Kalkulator {
```

```

// Atribut
int angka1;
int angka2;

// Method
int tambah() {
    return angka1 + angka2;
}
}

```

Main.java

```

package modul_2.bagian_2;

public class Main {
    public static void main(String[] args) {
        Kalkulator kalkulator = new Kalkulator();
        kalkulator.angka1 = 5;
        kalkulator.angka2 = 10;

        System.out.println("Hasil Penjumlahan: " + kalkulator.tambah());
    }
}

```

Analisa

Kode ini mendemonstrasikan penerapan objek sebagai unit komputasi yang memiliki kemampuan untuk memproses data melalui penggunaan method dengan nilai kembalian (return value). Pada class Kalkulator, didefinisikan dua atribut bertipe integer, yaitu angka1 dan angka2, yang berfungsi sebagai penyimpanan data mentah. Komponen terpenting dalam class ini adalah method int tambah(), yang secara spesifik menggunakan tipe data int (bukan void) pada deklarasinya. Hal ini menandakan bahwa method tersebut tidak hanya menjalankan perintah, tetapi juga bertanggung jawab untuk mengembalikan hasil operasi aritmatika kepada bagian program yang memanggilnya. Pada class Main, proses dimulai dengan instansiasi objek kalkulator untuk mengalokasikan memori, diikuti dengan pengisian nilai ke atribut secara langsung. Saat baris kalkulator.tambah() dijalankan di dalam perintah System.out.println, sistem akan masuk ke dalam logika penjumlahan di class Kalkulator dan "membawa pulang" hasil perhitungan tersebut untuk langsung ditampilkan ke layar. Pola ini menunjukkan efisiensi dalam aliran data, di mana kita tidak perlu membuat variabel tambahan di kelas utama untuk menampung hasil perhitungan sementara, melainkan langsung memanfaatkan perilaku (behavior) yang sudah tertanam di dalam objek tersebut.

3.3 bagian_3

AksesModifier.java

```

package modul_2.bagian_3;

public class AksesModifier {
    public int publicVar = 1;
    private int privateVar = 2;
    protected int protectedVar = 3;
}

```

```

int defaultVar = 4; // default

public void tampilkan() {
    System.out.println("Public: " + publicVar);
    System.out.println("Private: " + privateVar);
    System.out.println("Protected: " + protectedVar);
    System.out.println("Default: " + defaultVar);
}
}

```

Main.java

```

package modul_2.bagian_3;

public class Main {
    public static void main(String[] args) {
        AksesModifier contoh = new AksesModifier();
        contoh.tampilkan();

        // System.out.println(contoh.privateVar); // Error: privateVar tidak dapat diakses
    }
}

```

Analisa

Analisa Implementasi Access Modifier Kode ini mendemonstrasikan penerapan tingkat aksesibilitas (visibility) pada anggota kelas, yang merupakan aspek krusial dalam menjaga integritas data. Pada class `AksesModifier`, dideklarasikan empat jenis variabel dengan modifier yang berbeda: `public`, `private`, `protected`, dan `default`. Penggunaan modifier ini menentukan sejauh mana sebuah atribut dapat dilihat atau dimanipulasi oleh kelas lain. Dalam class ini, sebuah method publik bernama `tampilkan()` disediakan sebagai satu-satunya cara legal untuk mengakses dan mencetak nilai dari keempat variabel tersebut secara bersamaan dari dalam lingkup internal kelas. Pada class `Main`, analisis menunjukkan bahwa akses terhadap variabel `privateVar` akan memicu error jika dilakukan secara langsung dari luar kelas. Hal ini membuktikan bahwa modifier `private` berhasil mengunci data agar tidak bisa dijangkau oleh entitas luar, bahkan jika masih berada dalam package yang sama. Sementara itu, variabel lainnya seperti `publicVar` dan `defaultVar` masih dapat diakses karena batasan package-nya belum terlampaui. Implementasi ini menegaskan bahwa pemilihan modifier yang tepat sangat menentukan struktur keamanan sebuah aplikasi; penggunaan `private` pada atribut seringkali menjadi pilihan utama untuk memaksa akses melalui jalur resmi, yang dalam praktik selanjutnya akan diwujudkan melalui mekanisme setter dan getter.

3.4 bagian_4

Mobil.java

```

package modul_2.bagian_4;

public class Mobil {
    private String merk;

    // Setter

```

```

    public void setMerk(String merk) {
        this.merk = merk;
    }

    // Getter
    public String getMerk() {
        return merk;
    }
}

```

Main.java

```

package modul_2.bagian_4;

public class Main {
    public static void main(String[] args) {
        Mobil mobil = new Mobil();
        mobil.setMerk("Toyota");

        System.out.println("Merk Mobil: " + mobil.getMerk());
    }
}

```

Analisa

Kode ini menunjukkan penerapan prinsip enkapsulasi secara penuh melalui pembatasan akses langsung terhadap atribut kelas. Pada class Mobil, atribut merk dideklarasikan dengan modifier private, yang berarti variabel tersebut dikunci secara internal dan tidak dapat diakses atau dimodifikasi secara langsung dari luar kelas. Untuk menjembatani kebutuhan akses data, disediakan dua method publik, yaitu setMerk sebagai Setter dan getMerk sebagai Getter. Penggunaan kata kunci this di dalam Setter sangat krusial untuk menegaskan bahwa nilai yang diterima dari parameter method harus disimpan ke dalam variabel anggota milik objek tersebut, bukan variabel lokal. Di dalam class Main, manipulasi data dilakukan melalui jalur resmi yang telah disediakan. Baris mobil.setMerk("Toyota") menunjukkan proses pemberian nilai yang terkontrol, sementara mobil.getMerk() digunakan untuk memanggil kembali data tersebut untuk ditampilkan. Analisis terhadap struktur ini membuktikan keunggulan enkapsulasi dalam hal integritas data; pengembang memiliki kontrol penuh atas bagaimana data diubah atau dibaca. Pola ini memungkinkan penambahan logika validasi di masa depan di dalam method Setter (misalnya memastikan nama merk tidak kosong) tanpa harus mengubah kode yang ada di kelas utama, sehingga menciptakan sistem yang lebih fleksibel dan mudah dipelihara.

3.5 bagian_5

Person.java

```

package modul_2.bagian_5;

public class Person {
    private String nama;
    private int umur;

    // Default Constructor
}

```

```

public Person() {
    nama = "Unknown";
    umur = 0;
}

// Parameterized Constructor
public Person(String nama, int umur) {
    this.nama = nama;
    this.umur = umur;
}

// Method
public void tampilkanInfo() {
    System.out.println("Nama: " + nama);
    System.out.println("Umur: " + umur);
}
}

```

Main.java

```

package modul_2.bagian_5;

public class Main {
    public static void main(String[] args) {
        // Menggunakan Default Constructor
        Person person1 = new Person();

        // Menggunakan Parameterized Constructor
        Person person2 = new Person("Budi", 25);

        person1.tampilkanInfo();
        person2.tampilkanInfo();
    }
}

```

Analisa

Kode ini mendemonstrasikan fleksibilitas inisialisasi objek melalui penerapan Constructor Overloading pada class Person. Di dalam class tersebut, didefinisikan dua jenis constructor dengan parameter yang berbeda. Default Constructor berfungsi untuk menciptakan objek dengan nilai standar ("Unknown" dan 0), sehingga memastikan objek selalu memiliki keadaan awal yang valid meskipun tidak ada input dari pengguna. Sementara itu, Parameterized Constructor memungkinkan pengembang untuk langsung memetakan data nama dan umur ke atribut objek pada saat instansiasi menggunakan keyword this. Analisis pada class Main menunjukkan efisiensi dalam pembuatan objek sesuai kebutuhan konteks program. Penggunaan person1 yang menggunakan constructor tanpa parameter menghasilkan output nilai default, sedangkan person2 menunjukkan hasil inisialisasi langsung yang lebih ringkas. Penerapan overloading ini membuktikan keunggulan desain program yang adaptif, di mana sebuah class dapat menyediakan berbagai jalur pembuatan objek tanpa harus mengubah struktur dasar atributnya. Selain itu, penggunaan method tampilkanInfo() di sini berperan untuk merangkum proses penampilan data sehingga kode di program utama menjadi lebih bersih dan terorganisir.

3.6 bagian_6

Buku.java

```
package modul_2.bagian_6;

public class Buku {
    // Atribut (private)
    private String judul;
    private String pengarang;
    private int tahunTerbit;

    // Constructor (default)
    public Buku() {
        this.judul = "Unknown";
        this.pengarang = "Unknown";
        this.tahunTerbit = 0;
    }

    // Constructor (parameterized)
    public Buku(String judul, String pengarang, int tahunTerbit) {
        this.judul = judul;
        this.pengarang = pengarang;
        this.tahunTerbit = tahunTerbit;
    }

    // Setter dan Getter
    public void setJudul(String judul) {
        this.judul = judul;
    }

    public String getJudul() {
        return judul;
    }

    public void setPengarang(String pengarang) {
        this.pengarang = pengarang;
    }

    public String getPengarang() {
        return pengarang;
    }

    public void setTahunTerbit(int tahunTerbit) {
        this.tahunTerbit = tahunTerbit;
    }

    public int getTahunTerbit() {
        return tahunTerbit;
    }

    // Method untuk menampilkan informasi buku
    public void tampilkanInfo() {
        System.out.println("Judul: " + judul);
        System.out.println("Pengarang: " + pengarang);
        System.out.println("Tahun Terbit: " + tahunTerbit);
        System.out.println("-----");
    }
}
```


Main.java

```
package modul_2.bagian_6;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Perpustakaan perpustakaan = new Perpustakaan();
        int pilihan;

        do {
            // Menu
            System.out.println("\n=== Sistem Manajemen Perpustakaan ===");
            System.out.println("1. Tambah Buku");
            System.out.println("2. Tampilkan Semua Buku");
            System.out.println("3. Cari Buku");
            System.out.println("4. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = scanner.nextInt();
            scanner.nextLine(); // Membersihkan newline

            switch (pilihan) {
                case 1:
                    // Tambah Buku
                    System.out.print("Masukkan judul buku: ");
                    String judul = scanner.nextLine();
                    System.out.print("Masukkan nama pengarang: ");
                    String pengarang = scanner.nextLine();
                    System.out.print("Masukkan tahun terbit: ");
                    int tahunTerbit = scanner.nextInt();
                    scanner.nextLine(); // Membersihkan newline

                    Buku bukuBaru = new Buku(judul, pengarang, tahunTerbit);
                    perpustakaan.tambahBuku(bukuBaru);
                    break;

                case 2:
                    // Tampilkan Semua Buku
                    perpustakaan.tampilkanSemuaBuku();
                    break;

                case 3:
                    // Cari Buku
                    System.out.print("Masukkan judul buku yang dicari: ");
                    String judulCari = scanner.nextLine();
                    perpustakaan.cariBuku(judulCari);
                    break;

                case 4:
                    // Keluar
                    System.out.println("Terima kasih telah menggunakan sistem ini!");
                    break;

                default:
                    System.out.println("Pilihan tidak valid. Silakan coba lagi.");
            }
        } while (pilihan != 4);
    }
}
```

```

        scanner.close();
    }
}

```

Perpustakaan.java

```

package modul_2.bagian_6;

import java.util.ArrayList;

public class Perpustakaan {
    // Atribut (private)
    private ArrayList<Buku> daftarBuku;

    // Constructor
    public Perpustakaan() {
        daftarBuku = new ArrayList<>();
    }

    // Method untuk menambahkan buku
    public void tambahBuku(Buku buku) {
        daftarBuku.add(buku);
        System.out.println("Buku berhasil ditambahkan!");
    }

    // Method untuk menampilkan semua buku
    public void tampilkanSemuaBuku() {
        if (daftarBuku.isEmpty()) {
            System.out.println("Tidak ada buku dalam perpustakaan.");
        } else {
            System.out.println("Daftar Buku:");
            for (Buku buku : daftarBuku) {
                buku.tampilkanInfo();
            }
        }
    }

    // Method untuk mencari buku berdasarkan judul
    public void cariBuku(String judul) {
        boolean ditemukan = false;
        for (Buku buku : daftarBuku) {
            if (buku.getJudul().equalsIgnoreCase(judul)) {
                System.out.println("Buku ditemukan:");
                buku.tampilkanInfo();
                ditemukan = true;
                break;
            }
        }
        if (!ditemukan) {
            System.out.println("Buku dengan judul \"" + judul + "\" tidak ditemukan.");
        }
    }
}

```

Analisa

Analisa Implementasi Sistem Manajemen Perpustakaan Kode ini merupakan puncak dari praktikum yang mengintegrasikan seluruh konsep fundamental Pemrograman Berorientasi Objek ke dalam satu aplikasi fungsional. Pada class Buku, prinsip enkapsulasi diterapkan secara ketat dengan penggunaan atribut privat yang dikelola melalui Setter dan Getter. Penggunaan class Perpustakaan sebagai pengelola koleksi objek menunjukkan hubungan Composition, di mana sebuah objek besar menampung sekumpulan objek kecil dalam bentuk ArrayList. Analisis terhadap penggunaan ArrayList menunjukkan efisiensi dalam pengelolaan data dinamis, yang memungkinkan sistem untuk menambah, menyimpan, dan mencari referensi objek tanpa dibatasi oleh ukuran array statis. Pada class Main, implementasi antarmuka berbasis menu menggunakan perulangan do-while dan percabangan switch-case menciptakan alur interaksi pengguna yang terstruktur. Detil teknis yang sangat penting dalam kode ini adalah penanganan buffer pada class Scanner, di mana perintah scanner.nextLine() digunakan setelah nextInt() untuk membersihkan karakter newline yang tersisa, sehingga input teks selanjutnya tidak terlewat. Selain itu, logika pencarian pada method cariBuku yang menggunakan perbandingan equalsIgnoreCase membuktikan bahwa program telah dirancang untuk lebih fleksibel terhadap variasi input pengguna. Secara keseluruhan, sistem ini mendemonstrasikan bagaimana objek-objek yang berbeda dapat berinteraksi secara sinergis untuk memecahkan masalah manajemen data di dunia nyata.

4. Analisis Latihan Praktikum

4.1 Latihan_1

Buku.java

```
package modul_2.latihan.latihan_1;

public class Buku {
    String judul;
    String pengarang;
}
```

Main.java

```
package modul_2.latihan.latihan_1;

public class Main {
    public static void main(String[] args) {
        // 1. Membuat object dari class Buku
        Buku buku1 = new Buku();

        // 2. Mengisi nilai atribut (judul dan pengarang)
        buku1.judul = "Laskar Pelangi";
        buku1.pengarang = "Andrea Hirata";

        // 3. Menampilkan nilai atribut tersebut
        System.out.println("Judul Buku: " + buku1.judul);
        System.out.println("Pengarang : " + buku1.pengarang);
    }
}
```

```
}  
}
```

Analisa

Kode pada bagian latihan pertama ini mengulangi konsep fundamental mengenai pendefinisian blueprint dan proses instansiasi objek dalam lingkup yang lebih spesifik. Pada class Buku, analisis menunjukkan penggunaan atribut sederhana berupa judul dan pengarang dengan tingkat akses default (package-private). Struktur ini dirancang untuk menunjukkan fungsionalitas paling dasar dari sebuah kelas sebagai entitas penyimpan data (data holder) tanpa adanya proteksi enkapsulasi atau logika perilaku (behavior) tambahan. Pada class Main, alur eksekusi dilakukan secara prosedural namun tetap berbasis objek, dimulai dari pembuatan instance buku1 menggunakan keyword new. Analisis terhadap pengisian nilai atribut menunjukkan akses langsung ke variabel anggota objek, yang kemudian diteruskan ke perintah keluaran untuk ditampilkan ke konsol. Praktik pada latihan ini mempertegas pemahaman mengenai siklus hidup objek—mulai dari deklarasi kelas, alokasi memori saat pembuatan objek, hingga pemanfaatan data yang tersimpan di dalamnya—sebelum nantinya diintegrasikan dengan tingkat keamanan data yang lebih tinggi seperti akses modifier.

4.2 Latihan_2

Lingkaran.java

```
package modul_2.latihan.latihan_2;  
  
public class Lingkaran {  
    double jariJari;  
  
    // Method untuk menghitung luas lingkaran  
    public double hitungLuas() {  
        return 3.14 * jariJari * jariJari;  
    }  
}
```

Main.java

```
package modul_2.latihan.latihan_2;  
  
public class Main {  
    public static void main(String[] args) {  
        // 1. Membuat object dari class Lingkaran  
        Lingkaran bundar = new Lingkaran();  
  
        // 2. Mengisi nilai atribut jariJari  
        bundar.jariJari = 10;  
  
        // 3. Memanggil method hitungLuas() dan menyimpannya dalam variabel  
        double hasilLuas = bundar.hitungLuas();  
    }  
}
```

```

        // Menampilkan hasil
        System.out.println("Jari-jari lingkaran: " + bundar.jariJari);
        System.out.println("Luas lingkaran      : " + hasilLuas);
    }
}

```

Analisa

Kode ini mendemonstrasikan bagaimana sebuah objek tidak hanya berfungsi sebagai penyimpan data, tetapi juga memiliki kemampuan untuk melakukan perhitungan matematis secara mandiri. Pada class Lingkaran, didefinisikan satu atribut jariJari bertipe double untuk menampung nilai numerik dengan presisi desimal. Analisis pada method hitungLuas() menunjukkan penggunaan nilai kembalian (return value) yang mengolah data atribut jariJari menggunakan rumus luas lingkaran ($L = \pi \times r^2$). Dengan menempatkan rumus di dalam method, logika perhitungan terbungkus rapi di dalam kelas yang bersangkutan, sesuai dengan prinsip tanggung jawab tunggal dalam desain objek. Pada class Main, proses dimulai dengan instansiasi objek bundar dan pengisian nilai atribut secara langsung. Langkah selanjutnya menunjukkan pemanggilan method hitungLuas() yang hasilnya disimpan ke dalam variabel lokal hasilLuas sebelum ditampilkan. Analisis terhadap alur ini membuktikan efisiensi dalam penggunaan objek; kelas utama tidak perlu mengetahui bagaimana cara menghitung luas lingkaran secara detail, melainkan cukup meminta objek bundar untuk memberikan hasilnya. Pendekatan ini memudahkan pemeliharaan kode karena jika di masa depan terdapat perubahan nilai presisi π , perubahan tersebut cukup dilakukan satu kali di dalam class Lingkaran tanpa mengganggu logika di class Main.

4.3 Latihan_3

AkunBank.java

```

package modul_2.latihan.latihan_3;

public class AkunBank {
    // Atribut private hanya bisa diakses di dalam class ini sendiri
    private double saldo;

    // Constructor untuk mengisi saldo awal
    public AkunBank(double saldoAwal) {
        this.saldo = saldoAwal;
    }

    // Method public agar saldo bisa dilihat dari luar secara aman
    public void tampilkanSaldo() {
        System.out.println("Saldo saat ini: Rp" + saldo);
    }
}

```

Main.java

```

package modul_2.latihan.latihan_3;

```

```

public class Main {
    public static void main(String[] args) {
        AkunBank tabungan = new AkunBank(500000);

        // 1. Memanggil method public (Berhasil)
        tabungan.tampilkanSaldo();

        // 2. Mencoba akses atribut private secara langsung (Akan Error)
        // System.out.println(tabungan.saldo); // <--- Hapus komentar ini untuk melihat error
    }
}

```

Analisa

Kode ini berfokus pada penerapan konsep keamanan data melalui pembatasan akses atribut, yang merupakan inti dari prinsip enkapsulasi. Pada class AkunBank, atribut saldo dideklarasikan dengan modifier private. Analisis terhadap pemilihan modifier ini menunjukkan upaya untuk melindungi data sensitif agar tidak dapat dimanipulasi secara ilegal dari luar kelas, seperti pengubahan nilai saldo secara langsung tanpa melalui prosedur perbankan yang sah. Untuk menginisialisasi nilai tersebut, digunakan sebuah Parameterized Constructor yang secara otomatis mengisi saldo awal pada saat objek diciptakan. Pada class Main, interaksi dengan objek tabungan dilakukan melalui method publik tampilkanSaldo(). Analisis pada bagian komentar kode membuktikan bahwa upaya untuk mengakses atribut saldo secara langsung akan menyebabkan compile-time error. Hal ini mempertegas fungsi enkapsulasi dalam menciptakan batasan yang jelas antara bagian internal kelas yang tersembunyi dengan bagian luar yang boleh berinteraksi. Dengan struktur seperti ini, integritas data tetap terjaga karena setiap akses terhadap informasi saldo harus melalui jalur resmi (method publik) yang telah ditentukan oleh pengembang kelas tersebut.

4.4 Latihan_4

Mahasiswa.java

```

package modul_2.latihan.latihan_4;

public class Mahasiswa {
    private String nama;
    private String nim;

    // Setter untuk Nama
    public void setName(String nama) {
        this.nama = nama;
    }

    // Getter untuk Nama
    public String getName() {
        return nama;
    }

    // Setter untuk NIM
    public void setNim(String nim) {
        this.nim = nim;
    }
}

```

```

        // Getter untuk NIM
        public String getNim() {
            return nim;
        }
    }
}

```

Main.java

```

package modul_2.latihan.latihan_4;

public class Main {
    public static void main(String[] args) {
        // Membuat object dari class Mahasiswa
        Mahasiswa mhs = new Mahasiswa();

        // Menggunakan setter untuk mengisi nilai atribut
        mhs.setNama("Budi Santoso");
        mhs.setNim("220101234");

        // Menampilkan nilai menggunakan getter
        System.out.println("Nama Mahasiswa : " + mhs.getNama());
        System.out.println("NIM Mahasiswa : " + mhs.getNim());
    }
}

```

Analisa

Kode ini mendemonstrasikan penerapan standar industri dalam pengelolaan data identitas melalui mekanisme Setter dan Getter. Pada class Mahasiswa, atribut nama dan nim dideklarasikan menggunakan akses modifier private. Analisis terhadap struktur ini menunjukkan bahwa data tersebut sengaja disembunyikan agar tidak dapat dimodifikasi secara sembarangan dari luar kelas. Untuk berinteraksi dengan atribut privat ini, disediakan method publik sebagai jalur kendali resmi. Penggunaan kata kunci this di dalam setiap method Setter berperan vital untuk memastikan nilai parameter yang diterima disimpan ke dalam variabel instansi yang benar, menghindari ambiguitas penamaan variabel. Pada class Main, proses manipulasi data tidak lagi dilakukan melalui akses variabel secara langsung, melainkan melalui pemanggilan method setNama() dan setNim(). Analisis terhadap alur ini membuktikan keuntungan dari sisi fleksibilitas; jika di masa depan terdapat aturan validasi tertentu (misalnya NIM harus terdiri dari 9 karakter), logika tersebut dapat disisipkan langsung di dalam method Setter tanpa merusak kode yang sudah ada di kelas utama. Penggunaan Getter untuk menampilkan data ke layar memastikan bahwa pengambilan informasi tetap berjalan melalui prosedur yang terorganisir, menciptakan kode yang lebih rapi, aman, dan mudah untuk dikembangkan lebih lanjut dalam skala aplikasi yang lebih besar.

4.5 Latihan_5

Barang.java

```

package modul_2.latihan.latihan_5;

```

```

public class Barang {
    String namaBarang;
    double harga;

    // Default Constructor
    public Barang() {
        this.namaBarang = "Belum Ada Nama";
        this.harga = 0;
    }

    // Parameterized Constructor
    public Barang(String namaBarang, double harga) {
        this.namaBarang = namaBarang;
        this.harga = harga;
    }

    public void tampilkanBarang() {
        System.out.println("Nama Barang: " + namaBarang);
        System.out.println("Harga      : Rp" + harga);
        System.out.println("-----");
    }
}

```

Main.java

```

package modul_2.latihan.latihan_5;

public class Main {
    public static void main(String[] args) {
        // 1. Menggunakan Default Constructor
        Barang barang1 = new Barang();

        // 2. Menggunakan Parameterized Constructor
        Barang barang2 = new Barang("Laptop Gaming", 15000000);

        // Menampilkan informasi kedua objek
        System.out.println("Info Barang 1:");
        barang1.tampilkanBarang();

        System.out.println("Info Barang 2:");
        barang2.tampilkanBarang();
    }
}

```

Analisa

Kode ini mendemonstrasikan fleksibilitas dalam proses inisialisasi objek melalui penerapan Constructor Overloading pada class Barang. Analisis terhadap struktur kelas menunjukkan adanya dua cara berbeda untuk menciptakan objek. Default Constructor digunakan untuk menjamin bahwa setiap objek yang dibuat tanpa argumen tetap memiliki status awal yang jelas, yaitu dengan nilai "Belum Ada Nama" dan harga 0. Hal ini sangat penting dalam pengembangan aplikasi untuk mencegah munculnya nilai null atau data yang tidak terdefinisi yang dapat menyebabkan error di kemudian hari. Pada class Main, penggunaan kedua jenis constructor tersebut menunjukkan bagaimana program dapat

beradaptasi dengan ketersediaan data. Objek `barang1` merepresentasikan kondisi di mana data belum tersedia saat instansiasi, sedangkan `barang2` menunjukkan efisiensi penggunaan Parameterized Constructor untuk mengisi atribut `namaBarang` dan `harga` secara instan dalam satu baris kode. Analisis pada method `tampilkanBarang()` memperlihatkan prinsip modularitas, di mana logika penampilan informasi dipusatkan di dalam kelas asal. Pendekatan ini memastikan bahwa setiap objek `Barang` bertanggung jawab penuh atas cara datanya ditampilkan, sehingga kode di program utama tetap ringkas dan terfokus pada alur eksekusi aplikasi.

5. Hasil dan Pembahasan

Berdasarkan rangkaian praktikum yang telah dilaksanakan, hasil yang diperoleh menunjukkan bahwa penerapan prinsip Pemrograman Berorientasi Objek (OOP) secara signifikan meningkatkan struktur dan keamanan sebuah program. Pada tahap awal, objek hanya berfungsi sebagai wadah data sederhana, namun seiring dengan diterapkannya akses modifier dan enkapsulasi, objek bertransformasi menjadi unit yang lebih aman dan mandiri. Pembahasan utama dalam praktikum ini terletak pada bagaimana Setter dan Getter memberikan kontrol penuh kepada pengembang untuk menjaga integritas data, serta bagaimana Constructor Overloading memberikan fleksibilitas dalam inisialisasi objek. Integrasi akhir pada sistem manajemen perpustakaan membuktikan bahwa objek-objek yang terenkapsulasi dengan baik dapat dikelola secara dinamis menggunakan `ArrayList`. Hasil eksekusi program menunjukkan bahwa penggunaan method dengan nilai kembalian (`return value`) memungkinkan pemisahan yang jelas antara logika pemrosesan data dengan antarmuka pengguna. Hal ini mempermudah proses pencarian dan manipulasi data tanpa risiko merusak struktur data internal objek tersebut. Secara keseluruhan, pembahasan ini menegaskan bahwa OOP bukan sekadar teknik penulisan kode, melainkan sebuah arsitektur yang memastikan aplikasi lebih modular dan mudah dikelola dalam jangka panjang.

6. Kesimpulan

Dari hasil praktikum yang telah dilakukan, dapat ditarik beberapa kesimpulan utama sebagai berikut:

- **Modularitas Program:** Penggunaan class dan object berhasil memecah kompleksitas program menjadi komponen-komponen kecil yang lebih mudah dikelola dan dikembangkan secara terpisah.
- **Keamanan Data:** Implementasi akses modifier `private` yang dikombinasikan dengan method Setter dan Getter efektif dalam melindungi data sensitif dari modifikasi langsung yang tidak tervalidasi.
- **Efisiensi Inisialisasi:** Penggunaan Constructor (baik default maupun berparameter) mempermudah proses pembuatan objek dengan memastikan setiap objek memiliki status awal yang jelas dan valid sejak saat instansiasi.
- **Fleksibilitas Pengelolaan Data:** Penggunaan Collection seperti `ArrayList` untuk menampung objek terbukti lebih unggul dibandingkan array statis dalam menangani data yang jumlahnya berubah-ubah secara dinamis.

7. Saran

Untuk pengembangan lebih lanjut dalam praktikum maupun implementasi kode di masa mendatang, terdapat beberapa saran yang dapat dipertimbangkan:

- Penambahan Validasi Data: Pada method Setter, sebaiknya ditambahkan logika validasi (misalnya pengecekan nilai negatif pada harga atau umur) untuk memastikan data yang masuk ke dalam objek selalu memenuhi kriteria bisnis.
- Eksplorasi Pilar OOP Lainnya: Disarankan untuk mulai mengimplementasikan konsep Inheritance (pewarisan) dan Polymorphism pada studi kasus yang lebih luas untuk mengurangi redundansi kode pada kelas-kelas yang memiliki kemiripan sifat.
- Penanganan Eksepsi: Untuk meningkatkan ketahanan program (robustness), sebaiknya ditambahkan mekanisme Error Handling menggunakan try-catch, terutama saat menangani input pengguna melalui Scanner guna menghindari kegagalan program akibat tipe data yang tidak sesuai.